Simulations in R: Bias, consistency and Regression with different assumptions

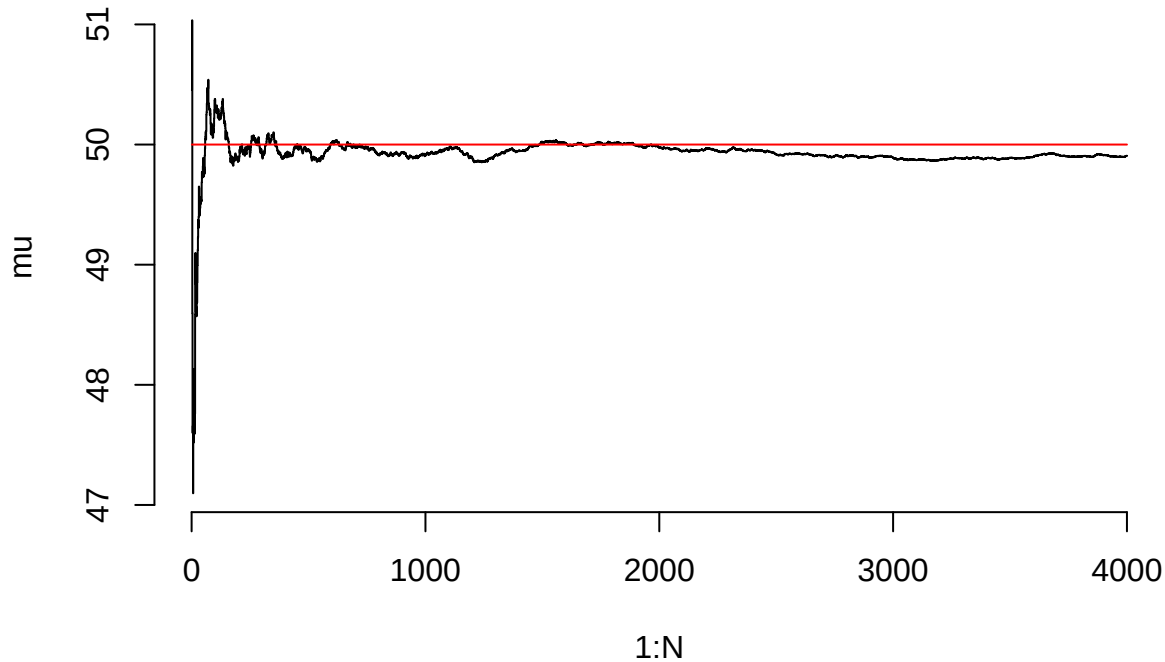
Innocent Maposa

2026-04-16

Mean bias

Let the population mean be $\mu = 50$ and standard deviation $\sigma = 5$. We can simulate data from this distribution and calculate mean:

```
N = 4000
set.seed(23435)
X = rnorm(N, mean = 50, sd = 5)
mu = rep(0, N)
for (n in 1:N) {
 mu[n] = sum(X[1:n])/n
}
plot(1:N, mu, type = 's', frame.plot = FALSE)
lines(1:N, rep(50,N), col = "red")
```



Shows that as n increases, the bias from true mean reduces. You can play around with different numbers. The same simulation approaches can be used to assess model properties under some specific assumption. The *tutorial below illustrate a simulation to assess effect of heteroscedasticity in regression analysis.*

Regression Assumptions

There are a number of assumptions that underpin linear regression models. Simulation can be a useful way of exploring these assumptions and understanding how violating these assumptions can lead to bias, large variance in the regression coefficients, and/or poor predictions.

Some typical assumptions include:

1. Homoskedasticity
2. Multicollinearity of independent variables
3. Measurement Error
4. Serial correlation

Today, we will explore the assumption of homoskedasticity.

Creating the baseline simulation

Before exploring how violations of the homoskedasticity assumption influence a regression model, we need a baseline model to compare it against. So, we will begin by simulating a simple linear regression with 1 predictor. Our model will look like this:

$y = 2 + 5 * x + e$

Where `e` will be random error from a normal distribution with a mean of 0 and standard deviation of 1.

```r
library(tidymodels)
```

```
## Warning: package 'tidymodels' was built under R version 4.5.2

## -- Attaching packages ------------------------------------- tidymodels 1.4.1 --

## v broom        1.0.12     v recipes      1.3.1
## v dials        1.4.2      v rsample      1.3.1
## v dplyr        1.1.4      v tailor       0.1.0
## v ggplot2      4.0.2      v tidyr        1.3.2
## v infer        1.0.9      v tune         2.0.1
## v modeldata    1.5.1      v workflows    1.3.0
## v parsnip      1.3.3      v workflowsets 1.1.1
## v purrr        1.1.0      v yardstick    1.3.2

## Warning: package 'broom' was built under R version 4.5.2

## Warning: package 'dials' was built under R version 4.5.2

## Warning: package 'ggplot2' was built under R version 4.5.2

## Warning: package 'infer' was built under R version 4.5.2

## Warning: package 'modeldata' was built under R version 4.5.2

## Warning: package 'parsnip' was built under R version 4.5.2

## Warning: package 'recipes' was built under R version 4.5.2

## Warning: package 'rsample' was built under R version 4.5.2

## Warning: package 'tailor' was built under R version 4.5.2

## Warning: package 'tidyr' was built under R version 4.5.2

## Warning: package 'tune' was built under R version 4.5.2

## Warning: package 'workflows' was built under R version 4.5.2

## Warning: package 'workflowsets' was built under R version 4.5.2

## Warning: package 'yardstick' was built under R version 4.5.2

## -- Conflicts ---------------------------------------- tidymodels_conflicts() --
## x purrr::discard() masks scales::discard()
## x dplyr::filter()  masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## x recipes::step()  masks stats::step()
```

```r
library(patchwork)

## set seed for reproducibility
set.seed(58968)

## create a data frame to store intercept values, slope values, their standard errors, and the model re
sim_params <- data.frame(intercept = NA,
                         slope = NA,
                         intercept_se = NA,
                         slope_se = NA,
                         model_rse = NA)

# true intercept value
intercept <- 2

# true slope value
slope <- 5

## Number of simulations to run
n <- 500

# random draw from a uniform distribution to simulate the independent variable
X <- runif(n = n, min = -1, max = 1)

## loop for regression model
for(i in 1:n){

  # create dependent variable, Y
  Y <- intercept + slope*X + rnorm(n = n, mean = 0, sd = 1)

  # build model
  model <- lm(Y ~ X)

  # # store predictions
  fitted_vals <- model$fitted.values

  # # store residuals
  # output_df[i, 2] <- model$residuals

  # variance-covariance matrix for the model
  vcv <- vcov(model)

  # estimates for the intercept
  sim_params[i, 1] <- model$coef[1]

  # estimates for the slope
  sim_params[i, 2] <- model$coef[2]

  # SE for the intercept
  sim_params[i, 3] <- sqrt(diag(vcv)[1])

  # SE for the slope
  sim_params[i, 4] <- sqrt(diag(vcv)[2])
```

```r
  # model RSE
  sim_params[i, 5] <- summary(model)$sigma

}

head(sim_params)
```

```
##   intercept    slope intercept_se   slope_se model_rse
## 1  1.966323 4.977753   0.04495439 0.07883016 1.0036923
## 2  1.947826 4.966831   0.04460191 0.07821206 0.9958226
## 3  2.111264 4.900025   0.04508832 0.07906500 1.0066825
## 4  2.029259 4.970287   0.04317960 0.07571795 0.9640667
## 5  1.976044 5.064834   0.04446964 0.07798012 0.9928694
## 6  1.977989 5.071687   0.04461868 0.07824146 0.9961969
```

Now we summarize the data to see if we have values close to the specified model parameters

```r
sim_params %>%
  summarize(across(.cols = everything(),
                   ~mean(.x)))
```

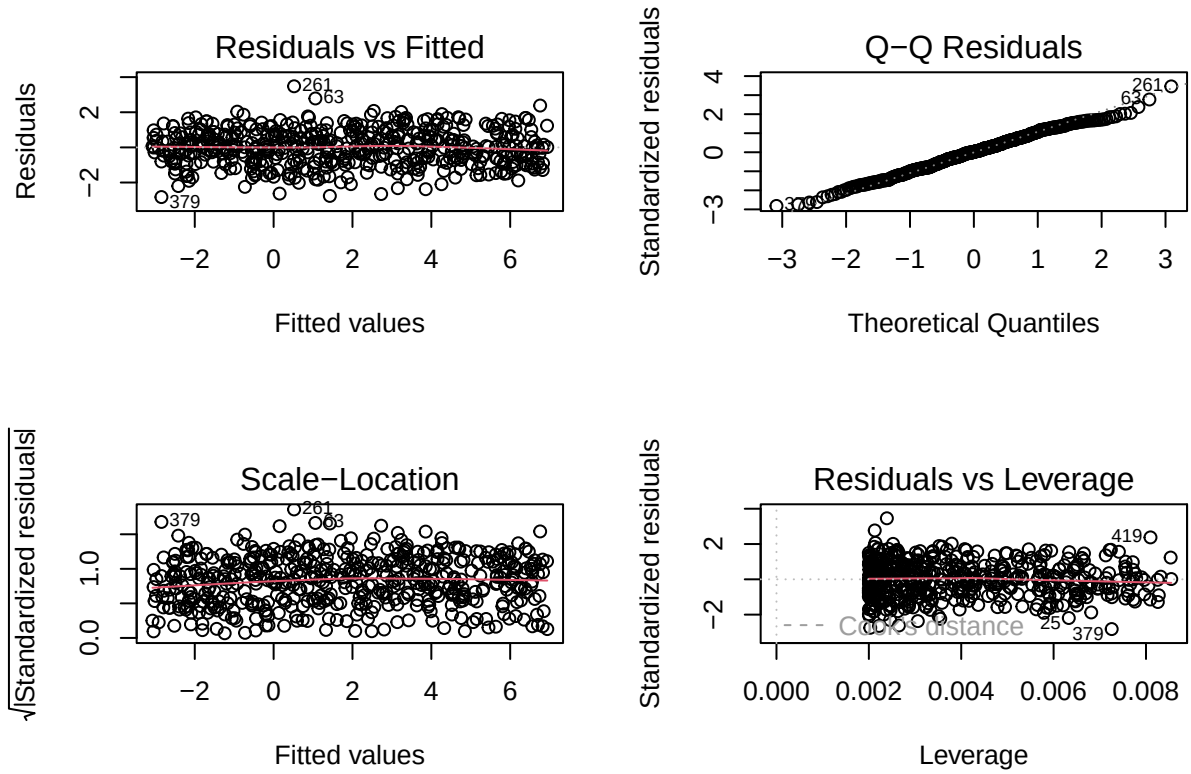
```
##   intercept    slope intercept_se   slope_se model_rse
## 1  1.999606 4.997908   0.04479056 0.07854287  1.000035
```

The final model of the 500 iterations is also stored from our for loop and we can look directly at it and create plots of the model fit.

```r
# model summary
summary(model)
```

```
##
## Call:
## lm(formula = Y ~ X)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.8278 -0.7432  0.0197  0.7272  3.4789
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.93654    0.04510   42.94   <2e-16 ***
## X            5.00945    0.07908   63.34   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.007 on 498 degrees of freedom
## Multiple R-squared:  0.8896, Adjusted R-squared:  0.8894
## F-statistic:  4012 on 1 and 498 DF,  p-value: < 2.2e-16
```

```r
# model fit plots
par(mfrow = c(2,2))
plot(model)
```



We can also create a function that lets us evaluate how often the 95% confidence interval of our simulated beta coefficients cover the true beta coefficients that we specified for the simulation. From there, we can get a coverage probability and a 95% probability coverage interval

```r
### Create a coverage probability function
coverage_interval95 <- function(beta_coef, se_beta_coef, true_beta_val, model_df){

  level95 <- 1 - (1 - 0.95) / 2

  # lower 95
  lower95 <- beta_coef - qt(level95, df = model_df)*se_beta_coef

  # upper 95
  upper95 <- beta_coef + qt(level95, df = model_df)*se_beta_coef

  # what rate did we cover the true value (hits and misses)
  hits <- ifelse(true_beta_val >= lower95 & true_beta_val <= upper95, 1, 0)
  prob_cover <- mean(hits)

  # create the probability coverage intervals
  low_coverage_interval <- prob_cover - 1.96 * sqrt((prob_cover * (1 - prob_cover)) / length(beta_coef)
```

```r
  upper_coverage_interval <- prob_cover + 1.96 * sqrt((prob_cover * (1 - prob_cover)) / length(beta_coe

  # results in a list
  return(list('Probability of Covering the True Value' = prob_cover,
              '95% Prob ability Coverage Intervals' = c(low_coverage_interval, upper_coverage_interval)

}
```

Let's apply it to the intercept.

```r
coverage_interval95(beta_coef = sim_params$intercept,
                    se_beta_coef = sim_params$intercept_se,
                    true_beta = intercept,
                    model_df = model$df.residual)
```

```
## $`Probability of Covering the True Value`
## [1] 0.946
##
## $`95% Prob ability Coverage Intervals`
## [1] 0.9261887 0.9658113
```

Now apply it to the slope

```r
coverage_interval95(beta_coef = sim_params$slope,
                    se_beta_coef = sim_params$slope_se,
                    true_beta = slope,
                    model_df = model$df.residual)
```

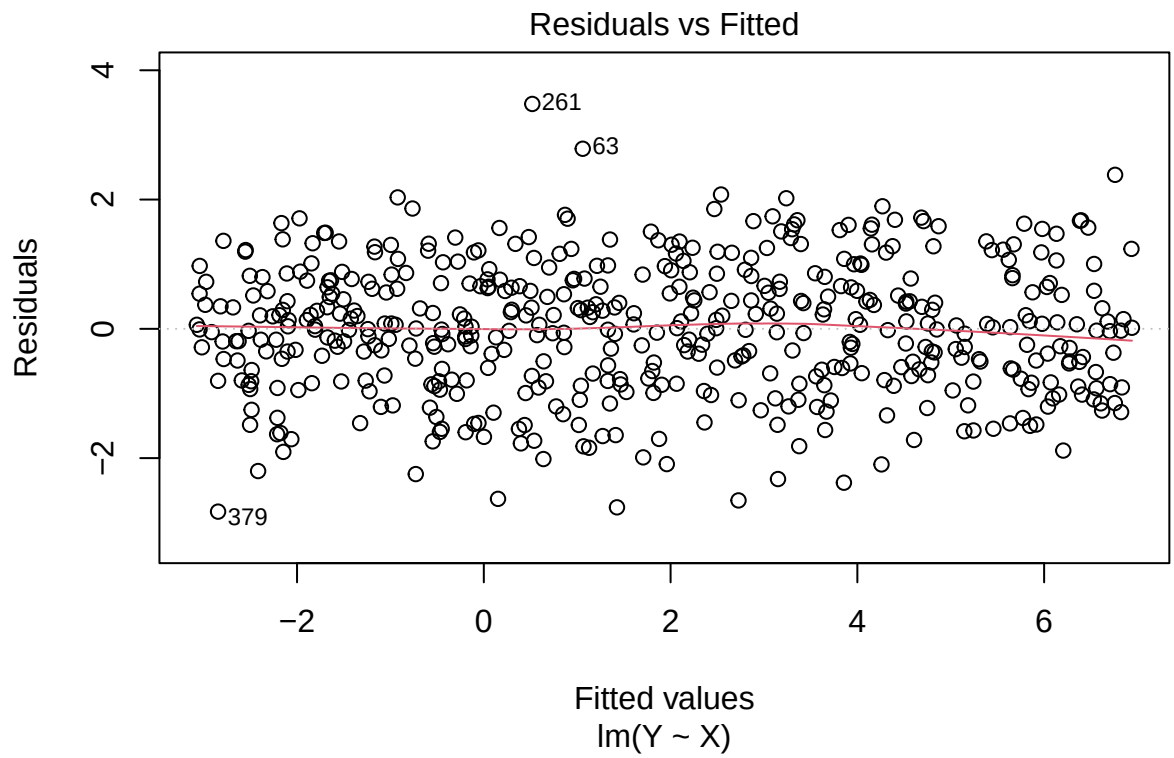
```
## $`Probability of Covering the True Value`
## [1] 0.966
##
## $`95% Prob ability Coverage Intervals`
## [1] 0.9501146 0.9818854
```

In both cases we are covering the true betas around 95% of the time, with relatively small intervals.

Homoskedasticity

Linear models make an assumption that the variance of the residuals remain constant across the predicted values (homoskedastic). We can see what this looks like by plotting the fitted values relative to the residuals, which was the first plot in the model check plots we created for the 500th simulation above. We can see that the residuals exhibit relatively the same amount of variance across the fitted values.

```r
plot(model, which = 1)
```



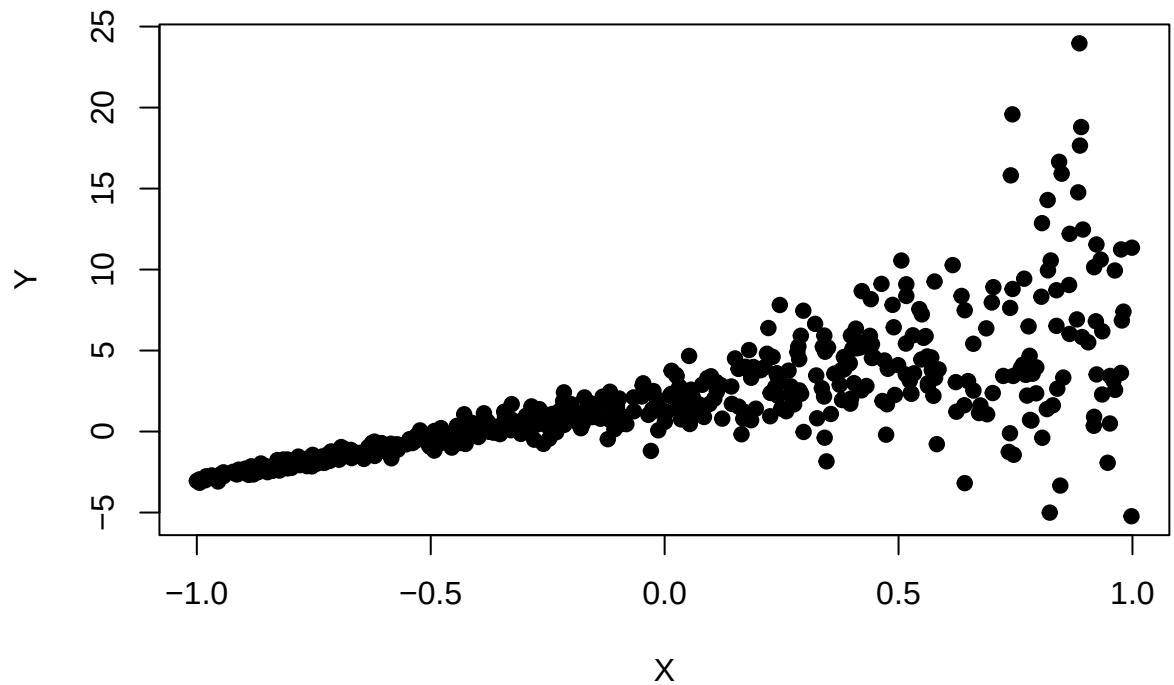
Let's simulate a model with heteroskedastic residuals and see what it looks like. We will keep the same intercept and slope parameters as above. The only thing will we do is add an exponential parameter to the error term of the model to create a heteroskedastic outcome in the residuals.

```
## parameter for heteroskedasticity
heteroskedasticity_param <- 2

## set seed for reproducibility
set.seed(22)

## data frame for results
heteroskedastic_sim_params <- data.frame(intercept = NA,
                       slope = NA,
                       intercept_se = NA,
                       slope_se = NA,
                       model_rse = NA)

## for loop
for(i in 1:n ){

  # the error variance of Y is a function of X plus some random noise
  Y <- intercept + slope*X + rnorm(n = n, mean = 0, sd = exp(X*heteroskedasticity_param))

  # model
  heteroskedastic_model <- lm(Y ~ X)
```

```r
  # variance-covariance matrix
  vcv <- vcov(heteroskedastic_model)

  # estimates for the intercept
  heteroskedastic_sim_params[i, 1] <- heteroskedastic_model$coef[1]

  # estimates for the slope
  heteroskedastic_sim_params[i, 2] <- heteroskedastic_model$coef[2]

  # SE for the intercept
  heteroskedastic_sim_params[i, 3] <- sqrt(diag(vcv)[1])

  # SE for the slope
  heteroskedastic_sim_params[i, 4] <- sqrt(diag(vcv)[2])

  # model RSE
  heteroskedastic_sim_params[i, 5] <- summary(heteroskedastic_model)$sigma

}

head(heteroskedastic_sim_params)
```

```
##   intercept    slope intercept_se  slope_se model_rse
## 1  1.884165 4.743169    0.0989576 0.1735279  2.209417
## 2  2.096654 5.246191    0.1156089 0.2027270  2.581189
## 3  1.950191 4.884566    0.1109784 0.1946072  2.477805
## 4  1.996244 5.062478    0.1135296 0.1990809  2.534765
## 5  2.048404 5.156628    0.1130880 0.1983065  2.524906
## 6  2.043749 4.992856    0.1283863 0.2251330  2.866469
```

```r
plot(X, Y, pch = 19)
```



The relationship between X and Y certainly looks weird given how it starts very tightly on the left side and then fans out on the right side.

Let's take the average across all 500 simulations for each coefficient and their corresponding standard errors.

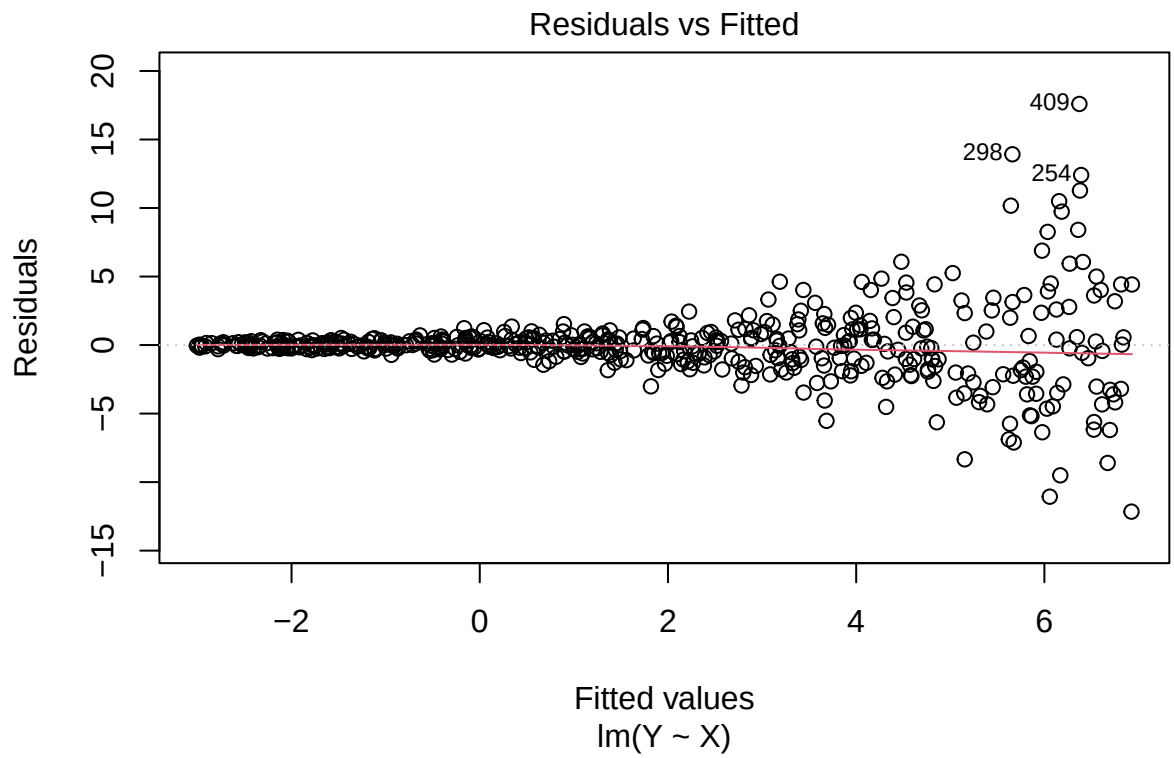
```
heteroskedastic_sim_params %>%
  summarize(across(.cols = everything(),
                   ~mean(.x)))
```

```
##   intercept    slope intercept_se  slope_se model_rse
## 1  2.001325 5.003661    0.1113449 0.1952499  2.485988
```

The coefficients of 2.0 for the intercept and 5 for the slope are exactly what we set them as for the simulation. However, notice how much larger the standard errors are for the intercept and slope compared to the original model above. Additionally, notice that the model residual standard error has increased substantially compared to the previous model.

Let's get the 500th model again and check out the fitted vs residual plot.

```
# fitted vs residuals
plot(heteroskedastic_model, which = 1)
```

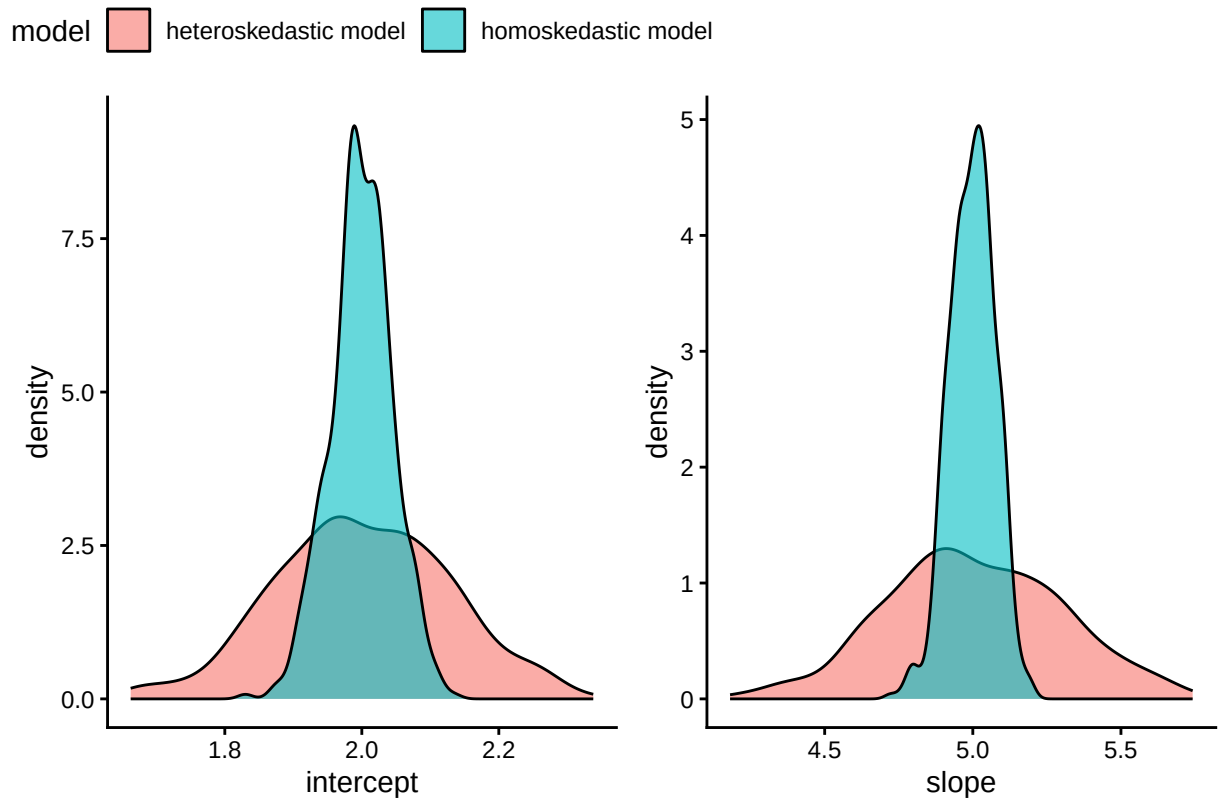


That looks like a large amount of heteroskedasticity as the residual variance is no longer homogenous across the range of fitted values. Notice the large fanning out towards the right side of the plot. As the predictions get larger so two does the variability in residuals, which we noticed when we plotted Y and X above.

What we've learned is that the estimate of intercept and slope is unbiased for both the heteroskedastic and homoskedastic models, as they both are centered on the parameters that we specified for the simulation (intercept = 2, slope = 5). However, the heteroskedastic model creates greater variance in our coefficients. We can visualize how much uncertainty there is under the heteroskedastic model relative to the homoskedastic model by visualizing the density of the coefficient estimates from our two model simulations.

```
plt_intercept <- sim_params %>%
  mutate(model = 'homoskedastic model') %>%
  bind_rows(
    heteroskedastic_sim_params %>%
      mutate(model = 'heteroskedastic model')
  ) %>%
  ggplot(aes(x = intercept, fill = model)) +
  geom_density(alpha = 0.6) +
  theme_classic() +
  theme(legend.position = "top")

plt_slope <- sim_params %>%
  mutate(model = 'homoskedastic model') %>%
  bind_rows(
    heteroskedastic_sim_params %>%
      mutate(model = 'heteroskedastic model')
  ) %>%
```

```r
  ggplot(aes(x = slope, fill = model)) +
  geom_density(alpha = 0.6) +
  theme_classic() +
  theme(legend.position = "none")

plt_intercept | plt_slope
```



Finally, let's see how often the 95% coverage interval is covering the true intercept and slope in the heteroskedastic model.

```r
coverage_interval95(beta_coef = heteroskedastic_sim_params$intercept,
                    se_beta_coef = heteroskedastic_sim_params$intercept_se,
                    true_beta = intercept,
                    model_df = model$df.residual)
```

```
## $`Probability of Covering the True Value`
## [1] 0.912
##
## $`95% Prob ability Coverage Intervals`
## [1] 0.8871681 0.9368319
```

```r
coverage_interval95(beta_coef = heteroskedastic_sim_params$slope,
                    se_beta_coef = heteroskedastic_sim_params$slope_se,
                    true_beta = slope,
                    model_df = model$df.residual)
```

```
## $`Probability of Covering the True Value`
## [1] 0.82
##
## $`95% Prob ability Coverage Intervals`
## [1] 0.7863245 0.8536755
```

Notice that we are no longer covering the true model values at the 95% level.